INTERVIEW PREPARATION PACK

Senior Data Analytics Analyst
Healthcare Claims Data

Cedar Gate Technologies — an IQVIA business

Burlington, MA · Hybrid, 3–4 days onsite · 41 questions with model answers

PART 1 — ROLE INTELLIGENCE (Read This First)

Who they are

Cedar Gate Technologies is a **value-based care (VBC) technology platform**, acquired by IQVIA. They serve four customer types: **payers** (health plans/BCBS), **providers** (health systems, ACOs, physician networks), **self-funded employers**, and **TPAs/service administrators**.

Their platform has three pillars sitting on one data foundation:

Pillar	What it does
Enterprise Data Management (EDM) / Data Lake	Ingests, normalizes, augments and enriches client data into a single source of truth. Handles 35+ data types in 1,500+ formats from 350+ suppliers.
Analytics	Value-Based Care Analytics + Healthcare Benefits Analytics. Descriptive, predictive, prescriptive. Bundled payment modeling to actuarial standards.
Population Health	Care management, care coordination, clinical decision support, gaps-in-care.
Payment Technology	Bundles Adjudication and Capitation Adjudication — converts fee-for-service claims into single bundled claims, prices episodes, distributes payments.

They also do **actuarial consulting** and **bundles program management** as managed services.

What this role ACTUALLY is

Read the JD carefully — this is **not** a BI/dashboard role. It is a **client data onboarding + data quality + implementation analyst** role. You sit between raw client data and the Cedar Gate Analytics Platform.

Your day is roughly: 1. A new client (or a new file feed) sends claims, eligibility, EMR, pharmacy, lab, or ancillary data 2. You profile it, map disparate fields into Cedar Gate's **unified schema** 3. You reconcile members across datasets (eligibility ↔ medical claims ↔ pharmacy ↔ EMR) 4. You standardize: relationship codes, demographics, member months, healthcare code sets 5. You run it through proprietary ETL, validate outputs against source-of-truth controls 6. When Client Success says "the client says their PMPM looks wrong," you're the one who root-causes it

The JD explicitly says: "This role assumes prior, hands-on experience working with complex healthcare claims datasets and does not include foundational training." That is a warning shot. They will test domain depth, not just SQL syntax.

Interview process (from candidate reports)

- Typically **recruiter screen → technical round → hiring manager → HR/culture**
- Average time to hire ~19 days
- Difficulty rated ~2.9/5 — described as fundamentals-focused rather than LeetCode-style
- Expect **behavioral ("tell me about a time")** plus a possible **case/scenario walkthrough**
- One reported flag worth noting: a candidate was asked about comfort with **unpaid overtime beyond 9–6**. Have a composed, non-defensive answer ready (see Part 6, Q-S10).

Your three-sentence positioning statement (memorize this)

> "I work in healthcare data where the hard part isn't the query — it's that every client sends the same concept in a different shape. I've spent [X] years profiling incoming claims and eligibility feeds, mapping them into a standard schema, and building the validation controls that catch problems before a client sees them. What I like about this role is that Cedar Gate's whole value proposition depends on that data foundation being right."

PART 2 — DOMAIN PRIMER (Non-Negotiable Knowledge)

If you know nothing else, know this section cold. Every technical answer below leans on it.

2.1 The claims lifecycle

```
Member enrolls (834 file) → Eligibility span created
   ↓
Member receives care → Provider bills
   ↓
837 claim submitted (P = professional/CMS-1500, I = institutional/UB-04, D = dental; NCPDP = pharmacy)
   ↓
Payer adjudicates → applies contract, benefits, edits
   ↓
835 remittance advice → payment + adjustment reason codes
   ↓
Claim may later be ADJUSTED, VOIDED, or REPLACED (this is where analysts get burned)
```

2.2 The X12 EDI transactions you must be able to name

Transaction	Purpose
834	Benefit enrollment & maintenance — the eligibility/membership file
837P / 837I / 837D	Claim submission — professional / institutional / dental
835	Electronic remittance advice (payment)
270 / 271	Eligibility inquiry / response
276 / 277	Claim status inquiry / response
NCPDP	Retail pharmacy claims standard (not X12)

2.3 Individual Relationship Codes (X12 5010, SBR02 / INS02)

The JD calls out **"relationship code alignment"** by name. Know these:

Code	Meaning
18	Self (the subscriber)
01	Spouse
19	Child
53	Life Partner
G8	Other Relationship
20	Employee
21	Unknown
39 / 40	Organ Donor / Cadaver Donor

Why it matters: clients send this field in wildly different forms — `"18"`, `"SELF"`, `"S"`, `"EE"`, `"M"` (member), `"1"`, blank. Alignment means mapping all variants to one canonical value, then validating the family structure: every dependent must roll up to exactly one subscriber, and there should be exactly one `Self` per subscriber ID per coverage period. Failures here break family-level reporting, dependent counts, and employer group billing.

2.4 Member months — the single most important denominator

Definition: one member enrolled for one month = one member month.

Why it exists: enrollment churns constantly (new hires, terminations, dependents aging off, mid-month adds). You cannot divide by "number of members" — you must divide by person-months of exposure.

How it's built: you explode each eligibility *span* (start date → end date) into one row per eligible month.

Business rules you must ask about (this is a great question to ask THEM): - **Mid-month enrollment:** count as full month, prorate by days, or use a "15th of the month" / "1st of the month" snapshot rule? - **Retro-terms:** a member terminated retroactively — do you restate history or leave it? - **Overlapping spans:** same member in two plans — dedupe to one member month or count both? - **Coverage type:** medical-only vs medical+Rx member months (Rx PMPM must use the Rx-eligible denominator, not the medical one)

Formulas:

```
PMPM = Total Cost ÷ Total Member Months

Utilization per 1,000 members per year
  = (Event Count ÷ Member Months) × 12,000

Contributive PMPM (service categories must sum to total):
  always divide each category by TOTAL population member months

Comparative PMPM (cohort vs cohort):
  divide each cohort by ITS OWN member months
```

That last distinction is a genuine senior-level differentiator. Say it out loud in the interview.

2.5 The money fields — never confuse these

```
BILLED (Charged)   = what the provider asked for (nearly meaningless analytically)
   ↓ minus contractual adjustment
ALLOWED            = contracted amount — THE analytic standard for cost/trend
   ↓ splits into
PAID (plan paid)   +   MEMBER COST SHARE (copay + coinsurance + deductible)
```

Rule of thumb: use **allowed** for cost trend and provider comparison (it's contract-normalized). Use **paid** when the question is "what did the plan spend." Use **billed** almost never. If an interviewer asks "which amount would you use for PMPM?" — the answer is "*depends on the question, but usually allowed, and I'd confirm with the client which they contractually report on.*"

2.6 Code sets

Code set	Domain	Notes
ICD-10-CM	Diagnoses	3–7 chars, alphanumeric. Watch decimal vs no-decimal formatting.
ICD-10-PCS	Inpatient procedures	7 chars, institutional only
CPT / HCPCS Level II	Professional procedures / supplies, drugs	CPT = 5 digits; HCPCS = letter + 4 digits
Revenue Codes	UB-04 institutional line detail	4-digit, leading-zero sensitive (`0450` = ER)
MS-DRG	Inpatient grouping/payment	~750 groups
NDC	Drugs	11-digit normalized (5-4-2); source often sends 10-digit — must zero-pad correctly
POS	Place of service	11 = office, 21 = inpatient, 23 = ER, 02/10 = telehealth
Type of Bill (TOB)	Institutional claim type	3-digit: facility + bill classification + frequency
NPI	Provider identifier	10-digit; billing vs rendering vs attending are different fields
Taxonomy	Provider specialty	

Classic trap: leading zeros. Revenue codes and NDCs get destroyed by Excel and by naive `INT` casting. Mention this unprompted — it signals real hands-on experience.

2.7 Risk adjustment & grouper concepts

- **CMS-HCC** — Medicare Advantage risk model
- **HHS-HCC** — commercial/ACA
- **CDPS** — Medicaid
- **Groupers:** ETG (Episode Treatment Groups), ACG (Johns Hopkins), CRG (3M), MARA (Milliman)
- **Why:** you cannot compare provider A's PMPM to provider B's without risk adjustment. Risk-adjusted PMPM = raw PMPM ÷ risk score. Cedar Gate explicitly enriches data with risk, quality, and SDOH (including Area Deprivation Index).

2.8 Claims lag / IBNR / completion

Claims take months to be submitted and paid. **Incurred But Not Reported (IBNR)** is the estimate of the missing tail. - **Incurred date** (service date) vs **paid date** (adjudication date) — the difference drives everything - **Completion factors:** a 3-month-old incurred month might be only 70% complete; a 12-month-old one 99% - **Practical rule:** never report a recent incurred month as final. Use a **runout period** (commonly 3 months for professional, longer for institutional) or apply completion factors.

If you say one thing that makes you sound senior, make it this: "*When a client says their PMPM dropped last quarter, my first check is always whether it's a real trend or just claims lag.*"

2.9 Value-based care vocabulary (Cedar Gate's home turf)

- **FFS vs VBC** — volume vs outcomes
- **Attribution** — assigning a member to a PCP/provider based on claims plurality or member selection
- **Bundled payment / episode** — trigger code, inclusion/exclusion logic, pre- and post-episode window, stop-loss
- **Capitation** — fixed PMPM payment regardless of utilization
- **Upside-only vs two-sided risk / shared savings**
- **Total Cost of Care (TCOC)**
- **MLR (Medical Loss Ratio)** — claims ÷ premium
- **Quality measures** — HEDIS, Stars, gaps in care

PART 3 — TECHNICAL QUESTIONS & ANSWERS

3A. SQL — Healthcare-Specific

Q1. "Write a query to calculate member months from an eligibility table."

Setup: `eligibility(member_id, coverage_start_dt, coverage_end_dt, plan_id, coverage_type)`

Answer — say the approach first, then write it:

> "Member months means exploding each eligibility span into one row per month, then deduping. I'll use a date-spine join, which is more portable and readable than recursion, and it plays well with Redshift."

```
WITH month_spine AS (
    SELECT DISTINCT DATE_TRUNC('month', datum) AS month_start
    FROM   dim_date
    WHERE  datum BETWEEN '2025-01-01' AND '2025-12-31'
),
member_month AS (
    SELECT DISTINCT              -- DISTINCT collapses overlapping spans
           e.member_id,
           m.month_start
    FROM   eligibility e
    JOIN   month_spine m
      ON   m.month_start <= e.coverage_end_dt
     AND   LAST_DAY(m.month_start) >= e.coverage_start_dt
    WHERE  e.coverage_type = 'MEDICAL'
)
SELECT month_start,
       COUNT(*) AS member_months
FROM   member_month
GROUP  BY month_start
ORDER  BY month_start;
```

Then add the senior-level commentary — this is what wins the round: - `DISTINCT` on (member_id, month) prevents double-counting a member enrolled in two overlapping plans - The overlap condition (`start <= span_end AND month_end >= span_start`) is the correct interval-intersection test — a naive `BETWEEN` on start date alone silently drops members - "This implements a *any-day-in-month counts* rule. Some clients want *enrolled on the 1st* or *enrolled on the 15th* or day-proration. I'd confirm the business rule before shipping." - Open-ended coverage: `COALESCE(coverage_end_dt, '9999-12-31')`

Q2. "Now calculate PMPM by service category."

```
WITH mm AS (
    SELECT COUNT(*) AS total_member_months
    FROM   member_month                      -- from Q1
    WHERE  month_start BETWEEN '2025-01-01' AND '2025-12-31'
),
spend AS (
    SELECT c.service_category,
           SUM(c.allowed_amt) AS allowed_amt
    FROM   claims c
    WHERE  c.service_start_dt BETWEEN '2025-01-01' AND '2025-12-31'
      AND  c.claim_status = 'PAID'
      AND  c.is_current_version = TRUE          -- critical
    GROUP  BY c.service_category
)
SELECT s.service_category,
       s.allowed_amt,
       mm.total_member_months,
       ROUND(s.allowed_amt / NULLIF(mm.total_member_months,0), 2) AS pmpm
FROM   spend s
CROSS  JOIN mm
ORDER  BY pmpm DESC;
```

Talking points: - Dividing every category by **total** member months makes categories sum to total PMPM (contributive PMPM) - `NULLIF` guards divide-by-zero - Anchored on **service date**, not paid date — so mention completion/lag - Filtered to current claim version — leads naturally into Q3

Q3. "A claim can be adjusted or voided. How do you make sure you're not double-counting?"

This is the #1 differentiator question in claims analytics. Answer:

> "Claims aren't immutable. A single service can appear as an original, then a replacement, then a void. If you just `SUM(allowed_amt)`, you'll overstate spend — sometimes dramatically."

Three patterns, depending on how the client sends data:

Pattern A — versioned claims (claim_id + version/sequence):

```
WITH ranked AS (
    SELECT c.*,
           ROW_NUMBER() OVER (
               PARTITION BY c.claim_id, c.claim_line_num
               ORDER BY c.version_num DESC, c.adjudication_dt DESC
           ) AS rn
    FROM   claims c
)
SELECT * FROM ranked WHERE rn = 1 AND claim_action_cd <> 'VOID';
```

Pattern B — offsetting reversals (a negative-dollar mirror row): don't filter — **let them net out**, but confirm the reversal and re-issue land in the same reporting period, otherwise you get a phantom dip in one month and a spike in the next.

Pattern C — Type of Bill frequency code (institutional): TOB position 3 tells you: `1` = admit-through-discharge (original), `7` = replacement of prior claim, `8` = void/cancel. Use it to sequence.

Close with: "*My validation control is a reconciliation report — distinct claim count and total allowed before and after dedupe, compared against the client's own paid-claims control total. If those don't tie, I go back before loading.*"

Q4. "Find members with a gap in coverage."

```
WITH spans AS (
    SELECT member_id,
           coverage_start_dt,
           coverage_end_dt,
           LAG(coverage_end_dt) OVER (
               PARTITION BY member_id ORDER BY coverage_start_dt
           ) AS prior_end_dt
    FROM   eligibility
)
SELECT member_id,
       prior_end_dt,
       coverage_start_dt,
       DATEDIFF(day, prior_end_dt, coverage_start_dt) - 1 AS gap_days
FROM   spans
WHERE  prior_end_dt IS NOT NULL
  AND  coverage_start_dt > DATEADD(day, 1, prior_end_dt)
ORDER  BY gap_days DESC;
```

Add: "Continuous enrollment is a prerequisite for most quality measures and trend cohorts — HEDIS typically allows one gap of ≤45 days. So this same logic feeds the continuously-enrolled flag."

Q5. "Identify 30-day all-cause readmissions."

```
WITH ip AS (
    SELECT member_id,
           claim_id,
           admit_dt,
           discharge_dt,
           LEAD(admit_dt) OVER (
               PARTITION BY member_id ORDER BY admit_dt
           ) AS next_admit_dt
    FROM   claims
    WHERE  claim_type = 'INSTITUTIONAL'
      AND  bill_type_cd LIKE '11%'          -- inpatient hospital
      AND  is_current_version = TRUE
)
SELECT member_id,
       claim_id      AS index_admission,
       discharge_dt,
       next_admit_dt,
       DATEDIFF(day, discharge_dt, next_admit_dt) AS days_to_readmit
FROM   ip
WHERE  next_admit_dt IS NOT NULL
  AND  next_admit_dt <= DATEADD(day, 30, discharge_dt);
```

Nuance to volunteer: "A true readmission measure excludes planned readmissions, transfers (where discharge status indicates transfer to another acute facility), obstetric and rehab stays, and deaths. Also, multiple claim lines per stay have to be rolled up into one stay first — otherwise every line looks like an admission."

Q6. "Find overlapping eligibility spans for the same member."

```
SELECT a.member_id,
       a.plan_id AS plan_a, a.coverage_start_dt, a.coverage_end_dt,
       b.plan_id AS plan_b, b.coverage_start_dt, b.coverage_end_dt
FROM   eligibility a
JOIN   eligibility b
  ON   a.member_id = b.member_id
 AND   a.elig_row_id < b.elig_row_id          -- avoid self & mirror pairs
 AND   a.coverage_start_dt <= b.coverage_end_dt
 AND   b.coverage_start_dt <= a.coverage_end_dt;
```

Why they care: overlaps inflate member months, which deflates PMPM. This is a standard pre-load data-quality check.

Q7. "Reconcile members across eligibility, medical claims, and pharmacy."

```
SELECT
    COUNT(DISTINCT e.member_id)                                    AS elig_members,
    COUNT(DISTINCT m.member_id)                                    AS med_claim_members,
    COUNT(DISTINCT CASE WHEN e.member_id IS NULL
                        THEN m.member_id END)                      AS orphan_claim_members,
    COUNT(DISTINCT CASE WHEN r.member_id IS NULL
                        THEN e.member_id END)                      AS no_rx_members
FROM        eligibility e
FULL OUTER JOIN (SELECT DISTINCT member_id FROM medical_claims)  m ON e.member_id = m.member_id
LEFT  JOIN      (SELECT DISTINCT member_id FROM pharmacy_claims) r ON e.member_id = r.member_id;
```

Frame it: "**Orphan claims** — claims for members with no eligibility record — are the number one onboarding red flag. Causes are usually: eligibility file has shorter history than claims, member ID formats differ between the claims system and the eligibility system, or the client sent a subscriber ID where a member ID was expected. I quantify it as a % of total allowed dollars, because 2% of members might be 15% of spend."

Q8. "Rank the top 1% highest-cost members and show their share of spend."

```sql
WITH member_spend AS (
    SELECT member_id, SUM(allowed_amt) AS total_allowed
    FROM   claims
    WHERE  service_start_dt BETWEEN '2025-01-01' AND '2025-12-31'
      AND  is_current_version = TRUE
    GROUP  BY member_id
),
pct AS (
    SELECT member_id, total_allowed,
           PERCENT_RANK() OVER (ORDER BY total_allowed) AS pr
    FROM   member_spend
)
SELECT COUNT(*)                                    AS top_1pct_members,
       SUM(total_allowed)                          AS top_1pct_spend,
       SUM(total_allowed) * 100.0 /
           (SELECT SUM(total_allowed) FROM member_spend) AS pct_of_total_spend
FROM   pct
WHERE  pr >= 0.99;
```

Add the domain insight: "Healthcare cost is extremely right-skewed — typically the top 1% of members drive 20–30% of spend, and the top 5% drive around half. That's why mean PMPM is fragile and why actuaries often truncate or pool high-cost claimants above a threshold when measuring trend."

Q9. Classic SQL fundamentals they may still ask

Question	Sharp answer
`WHERE` vs `HAVING`	`WHERE` filters rows pre-aggregation; `HAVING` filters groups post-aggregation.
`RANK` vs `DENSE_RANK` vs `ROW_NUMBER`	`RANK` skips after ties (1,1,3); `DENSE_RANK` doesn't (1,1,2); `ROW_NUMBER` never ties. Use `ROW_NUMBER` for dedupe.
`UNION` vs `UNION ALL`	`UNION` dedupes (expensive sort); `UNION ALL` doesn't. Use `ALL` unless you *need* dedupe.
Inner vs left join risk	A left join to a table with duplicate keys **fans out** rows and silently inflates `SUM()`. Always verify grain before joining.
`COUNT(*)` vs `COUNT(col)`	`COUNT(col)` ignores NULLs.
CTE vs subquery vs temp table	CTEs for readability; temp tables when reused multiple times or when you want stats/distribution control in Redshift.
Find duplicates	`GROUP BY key HAVING COUNT(*) > 1`
Delete duplicates keeping one	`ROW_NUMBER()` in a CTE, delete where `rn > 1`

3B. AWS & Redshift

Q10. "How do you optimize a slow Redshift query on a large claims table?"

Answer in layers — this shows structured thinking:

1. **Diagnose first:** `EXPLAIN` for the plan; `SVL_QUERY_REPORT` / `STL_ALERT_EVENT_LOG` for actual behavior. Look for the classic alerts: *very selective filter*, *missing statistics*, *nested loop join*, *distribution required (DS_BCAST_INNER)*.
2. **Distribution style:** - `DISTKEY` on the most common large-table join column — for claims, usually `member_id` (co-locates claims with eligibility) - `DISTSTYLE ALL` for small dimension tables (code lookups, provider dim) so they're replicated to every node - `EVEN` when there's no good key or the key is skewed - Check skew: if one slice holds most rows, performance collapses
3. **Sort keys:** compound sort key on the columns you filter by most — for claims, typically `(service_start_dt, ...)`. Date-range filters then use zone maps to skip whole blocks.
4. **Maintenance:** `VACUUM` (reclaim + re-sort) and `ANALYZE` (refresh stats). Stale stats are the most common cause of "it was fast last month."
5. **Column encoding:** AZ64 for numeric/date, ZSTD for text — reduces I/O, which is the real bottleneck.
6. **Query rewrite:** push predicates down, avoid `SELECT *` (columnar store — only read the columns you need), replace correlated subqueries with joins, pre-aggregate before joining.
7. **Platform:** WLM queues / concurrency scaling for mixed workloads; materialized views for repeated aggregations.

Q11. "What's the gotcha with primary keys in Redshift?"

> "**Redshift does not enforce primary key, unique, or foreign key constraints.** They're declarative only — the optimizer uses them for planning, but nothing stops duplicate rows from loading. So in a claims pipeline you must enforce uniqueness yourself: dedupe in the staging layer before the merge, and run a post-load uniqueness assertion. I've seen teams assume the constraint was doing work and end up with duplicated claim lines inflating PMPM."

That answer alone will land you as experienced.

Q12. "Walk me through loading a large claims file from S3."

```
COPY stg_claims
FROM 's3://client-bucket/claims/2025/11/'
IAM_ROLE 'arn:aws:iam::...:role/RedshiftLoadRole'
DELIMITER '|'
GZIP
DATEFORMAT 'YYYYMMDD'
BLANKSASNULL
EMPTYASNULL
MAXERROR 0
COMPUPDATE OFF
STATUPDATE OFF;
```

Best practices to mention: - **Split files into multiples of the number of slices** so every slice loads in parallel — one giant file = one slice doing all the work - **Compress (gzip/bzip2)** to cut transfer time - `MAXERROR 0` for financial data — fail loudly, don't silently skip rows - Inspect `STL_LOAD_ERRORS` on failure - **Load to staging → validate → merge to production** inside a transaction, never straight to prod

Q13. "How do you do an upsert (MERGE) in Redshift?"

```
BEGIN;

DELETE FROM prod_claims p
USING  stg_claims s
WHERE  p.claim_id = s.claim_id
  AND  p.claim_line_num = s.claim_line_num;

INSERT INTO prod_claims
SELECT * FROM stg_claims;

COMMIT;
```

> "Newer Redshift supports a native `MERGE`, but the delete-then-insert-in-a-transaction pattern is the durable one and works everywhere. The transaction matters — if it fails midway you must not be left with deleted-but-not-reinserted claims."

Q14. "Which AWS services would you expect in this stack?"

- **S3** — landing zone for client files, data lake storage, and the archive of every raw file received (immutable audit trail)
- **Redshift** — analytic warehouse; **RA3 nodes** separate compute from managed storage
- **Redshift Spectrum** — query cold historical data directly in S3 without loading it
- **Glue** — catalog + ETL jobs
- **Lambda** — event-driven triggers (file lands in S3 → kick off validation)
- **Step Functions** — orchestration of multi-stage pipelines
- **KMS** — encryption at rest; critical for PHI
- **CloudWatch** — pipeline monitoring and alerting
- **EMR** — Spark for very large transformations (note: in healthcare, "EMR" is ambiguous — clarify Electronic Medical Record vs Elastic MapReduce; making that joke lightly shows fluency)

Q15. "How do you handle PHI in a cloud data environment?"

- **HIPAA minimum necessary** — only pull the fields the analysis requires
- Encryption **in transit** (TLS/SFTP) and **at rest** (KMS)
- **Role-based access control**; separate prod from dev; no PHI in dev/test — use de-identified or synthetic data

- **De-identification:** HIPAA Safe Harbor (18 identifiers removed, dates limited to year, ages 90+ aggregated) vs Expert Determination
- **Audit logging** of every access
- Never email PHI, never put it in a ticket, never move it to a laptop
- **BAAs** with vendors and subcontractors
- If you ever spot exposed PHI: stop, don't propagate, escalate to security/privacy immediately

---

3C. ETL, Data Quality & Schema Mapping

Q16. "A new client's data arrives. Walk me through your process."

Use this seven-step frame — it maps almost perfectly onto the JD bullets:

1. **Requirements & documentation** — get the client's data dictionary, file layouts, code sets, and their own control totals (member count, claim count, total paid by month). Confirm history depth, refresh cadence, and delivery mechanism.
2. **Profile before you transform** — row counts, null rates, distinct value counts, min/max dates, value distributions per column. Never write a mapping spec off a data dictionary alone; dictionaries lie, data doesn't.
3. **Map to the canonical schema** — field-by-field mapping document with source field, target field, transformation logic, and defaulting rules. Flag every unmapped source field and every unfilled target field.
4. **Standardize** — relationship codes, gender/demographics, date formats, ID formats (leading zeros!), code sets (ICD/CPT/rev codes/NDC normalization), amount sign conventions.
5. **Reconcile & validate** — member counts vs their control totals, total allowed/paid by incurred month vs their financials, orphan claim rate, duplicate rate, referential integrity across eligibility/medical/Rx.
6. **Load** — staging → validation gate → production, with the raw file archived immutably.
7. **Monitor** — automated file-arrival checks, row-count variance vs trailing average, dollar variance thresholds, and alerting. Then document the whole thing so it's runnable by someone else.

Land it: "*The discipline I'd bring is: nothing loads to production until reconciliation ties to a client-provided control total. If it doesn't tie, that's a conversation with the client, not a workaround in the code.*"

Q17. "Client A sends gender as M/F, Client B as 1/2, Client C as MALE/FEMALE/U. How do you handle it?"

> "I don't hard-code the logic — I drive it from a **crosswalk/reference table** keyed by client and source system. Something like `(client_id, source_field, source_value, canonical_value, effective_date)`. Three reasons: new clients are a data change instead of a code change; the mapping is auditable and reviewable by non-engineers; and unmapped values can be detected automatically instead of silently defaulting.
>
> The critical rule is **fail loudly on unmapped values**. If client D sends `'X'` and we silently map anything unknown to `Unknown`, we've lost information and nobody knows. I'd route unmapped values to an exception report that gets reviewed before load. Same pattern applies to relationship codes, which is a bigger deal because family structure depends on it."

Q18. "What data quality checks do you build?"

Group them into four tiers — structure the answer, don't list randomly:

Tier	Checks
Structural	File arrived on schedule; expected columns present; row count within tolerance of trailing average; header/trailer record counts match
Field-level	Data types valid; dates parse and are plausible (no service dates in the future, no DOB in 1899); required fields non-null; IDs match expected format/length; amounts numeric
Referential	Every claim has an eligible member; every member has ≥1 eligibility span; every claim has a valid provider; diagnosis and procedure codes exist in the reference code set for that date of service
Business/statistical	PMPM within X% of prior period; member months within tolerance; duplicate claim rate; orphan claim %; gender/age distribution stable; service category mix stable; % of claims with $0 allowed

Then say: "*I separate hard-fail checks that block a load from soft-warn checks that flag for review, because if everything blocks, people start bypassing the gate.*"

Q19. "How do you handle slowly changing dimensions for members?"

"Member demographics change — address, plan, PCP, even name and gender. For anything that affects historical reporting I use **Type 2**: effective_start_dt, effective_end_dt, and a current-record flag, so a claim from March 2024 joins to the member's March 2024 attributes rather than today's. For attributes where only the current value matters, Type 1 overwrite is fine and cheaper. The decision is driven by whether the client ever needs to reproduce a prior report exactly — and in healthcare, for audit and for VBC contract settlement, they usually do."

Q20. "How do you reconcile members across datasets when there's no common key?"

"Layered matching. First **deterministic** on the strongest available identifiers — member ID, then subscriber ID + relationship + DOB, then SSN if present. Whatever survives goes to a **probabilistic/fuzzy** pass scoring across name (with phonetic normalization), DOB, gender, and address, with a threshold above which we auto-link, a middle band that goes to manual stewardship review, and a floor below which we reject.

Before any of that you must **normalize**: uppercase, strip punctuation, standardize addresses, handle nicknames and hyphenated surnames, handle transposed DOB digits.

The two failure modes are asymmetric. A **split** — one person appearing as two members — understates their cost and breaks continuity. An **overlay** — two people merged into one — is a patient-safety issue and a HIPAA issue. So I bias thresholds conservatively toward split-over-overlay and route the ambiguous band to a human. The output is a persistent master ID with full lineage so merges can be unwound."

3D. Excel & Statistics

Q21. "Which advanced Excel functions do you use?"

- **XLOOKUP** / `INDEX+MATCH` (bidirectional, not fragile to column insertion, handles not-found)
- **SUMIFS / COUNTIFS / AVERAGEIFS** with multiple criteria
- **Power Query** — the real answer for repeatable transformation: connect, unpivot, merge, append, refresh. "This is where I'd do file prep rather than manual formulas, because it's auditable and repeatable."
- **PivotTables + Power Pivot/DAX** — data model relationships, measures, `CALCULATE`
- **TEXT functions** — `TEXT`, `LEFT/RIGHT/MID`, `TRIM`, `SUBSTITUTE`, `TEXTSPLIT`
- **LET / LAMBDA** for readable complex logic
- **Conditional formatting + Data Validation** for QA workbooks

Volunteer the war story: "*The one thing I'm careful about is Excel destroying healthcare identifiers — it'll strip leading zeros off a revenue code, turn an NDC into scientific notation, and convert `1-9` into a date. I import as text via Power Query rather than double-clicking a CSV.*"

Q22. "How do you detect outliers in claims data?"

"First I distinguish **data errors** from **legitimate extremes**, because healthcare has genuine million-dollar claims. Techniques: IQR fences (1.5× or 3× IQR), z-scores for roughly normal distributions, and percentile capping. But cost is heavily right-skewed, so I prefer **log transformation** or **robust methods (median/MAD)** over mean-and-SD.

Practically, the tells for a data error are: allowed > billed, negative amounts without a matching reversal, unit counts that are implausible (30 knee replacements on one line), service date after paid date, or a single claim that's a large share of an entire month.

For trend analysis, actuaries commonly **truncate or pool** claims above a threshold — say $100K — so one transplant doesn't create a fake trend. I'd never delete the outlier from source; I'd flag it and handle it in the analytic layer."

Q23. "Mean or median for healthcare cost?"

"Median for describing a *typical* member, because the distribution is severely right-skewed — most members cost very little and a small tail costs enormous amounts. But **mean is what you need for budgeting and PMPM**, because total dollars is what gets paid. So I usually report both, plus the distribution shape and the high-cost claimant concentration. If someone only shows a mean PMPM without the distribution, they're hiding the risk."

Q24. "How would you tell whether a cost reduction is real?"

- Compare against a **control/comparison group**, not just prior period
- **Risk-adjust** — the population may simply have gotten healthier or the sick members disenrolled
- Account for **claims lag** — the most recent period always looks cheap
- Check **enrollment mix shift** (age/gender/plan design changes)
- Check **benefit design changes** (higher deductible shifts cost to members — plan paid drops, allowed doesn't)
- Test **statistical significance** and be honest about sample size
- Watch for **regression to the mean** — high-cost cohorts naturally get cheaper next year regardless of intervention

That last one is a genuinely senior observation in population health. Say it.

PART 4 — HEALTHCARE DOMAIN QUESTIONS

Q25. "What's the difference between professional and institutional claims?"

	Professional (837P / CMS-1500)	Institutional (837I / UB-04)
Billed by	Physicians, clinicians, labs, DME	Hospitals, SNFs, home health, hospice
Procedure coding	CPT / HCPCS	Revenue codes + CPT/HCPCS + ICD-10-PCS
Key fields	Place of Service, rendering NPI, modifiers	Type of Bill, revenue code, discharge status, admit/discharge dates, DRG
Grain	Service line per date	Line per revenue code within a stay
Analytic implication	One line ≈ one service	Must roll many lines up into one **stay/encounter**

Killer add-on: "*An inpatient stay generates both — the facility bills 837I and the surgeon and anesthesiologist bill 837P separately. If you count 'admissions' from professional claims you'll triple count. Episode and bundle logic has to stitch them together.*"

Q26. "What is a member month and why does it matter?"

Covered in 2.4. Deliver: definition → why churn makes headcount wrong → the formula → then immediately raise the mid-month/retro-term business rules. Raising the edge cases unprompted is the signal.

Q27. "What's the difference between a subscriber and a member?"

> "The **subscriber** is the person who holds the policy — usually the employee. **Members** are everyone covered under it: the subscriber plus dependents. Relationship code 18 identifies the subscriber. This matters for three reasons: member counts vs contract/subscriber counts are different denominators and employers often quote the wrong one; some clients send only subscriber IDs on claims, so dependent claims get misattributed to the subscriber; and family-level analysis requires clean subscriber-to-dependent linkage. It's also why relationship code standardization is called out in this JD."

Q28. "How do you know if a claim is paid, denied, or pending?"

- **Claim status** field, plus payment amount = 0 with a denial reason
- **835 CARC/RARC codes** — Claim Adjustment Reason Codes and Remark Codes explain *why* an amount wasn't paid
- **CAS segment group codes:** `CO` = contractual obligation (write-off), `PR` = patient responsibility, `OA` = other, `PI` = payer initiated
- **Analytic rule:** denied claims are usually excluded from cost, but **kept** for utilization/access analysis and for denial-rate reporting. Never silently drop them — a spike in denials is itself a finding.

Q29. "What EMR data would you expect, and how does it differ from claims?"

	Claims	EMR
Purpose	Billing	Clinical care
Strength	Complete across all providers a member visits; has cost	Clinical depth — labs, vitals, meds, problem lists, notes
Weakness	No clinical results, no severity, coding driven by reimbursement	Only one provider's view; no cost; messy, unstructured, inconsistent
Standards	X12, ICD/CPT	HL7 v2, FHIR, CCDA, LOINC, SNOMED, RxNorm

The synthesis point: "*Claims tell you what was billed; EMR tells you what actually happened clinically. Value-based care needs both — you can't close a diabetes gap-in-care from claims alone because you need the A1c result, not just that a test was ordered. That's exactly why a platform ingests both and links them at the member level, and why identity resolution across the two is such a hard problem.*"

Q30. "What is value-based care and why does it matter here?"

> "It's the shift from paying for volume to paying for outcomes. In fee-for-service a provider earns more by doing more. In VBC — shared savings, bundles, capitation — the provider takes on financial risk for a population or an episode and is rewarded for keeping cost down and quality up.
>
> It matters for this role because VBC contracts only work if the data is right. Attribution decides which members are yours. Member months decide the denominator. Risk scores decide whether your benchmark is fair. Episode logic decides what's inside the bundle. If any of those inputs are wrong, real money moves incorrectly — and a provider might take a loss on a contract because of a data mapping error. That's a level of accountability I find motivating rather than intimidating."

That answer directly mirrors Cedar Gate's positioning. Use it.

Q31. Quick-fire domain terms to have ready

HEDIS, Stars, MLR, TCOC, ACO, MA (Medicare Advantage), TPA, ASO, stop-loss, capitation, attribution, gaps in care, SDOH, Area Deprivation Index, IBNR, runout, DRG, HCC, RAF score, prior authorization, formulary, generic dispensing rate.

PART 5 — BEHAVIORAL QUESTIONS (STAR)

Use **STAR**: Situation (15%) → Task (10%) → Action (60%) → Result (15%, quantified). Keep answers to ~2 minutes.

Prepare 6 core stories and flex them across questions: 1. A nasty data quality problem you found and fixed 2. A client-facing situation where you delivered bad news or pushed back 3. A time you were wrong / made an error 4. A time you handled competing priorities or a hard deadline 5. A time you translated technical findings for a non-technical audience 6. A process or automation you built that outlasted you

---

Q32. "Tell me about yourself."

Structure: **Present → Past → Why here.** 90 seconds max.

> "I'm currently a [role] focused on healthcare data — primarily claims and eligibility, working in SQL and [Redshift/AWS] to onboard and validate client data. Before that I [prior]. The through-line in my work is data integrity: I've spent most of my career on the unglamorous part where messy source data becomes something people can safely make financial decisions on. I'm drawn to this role because Cedar Gate's whole platform depends on that layer being right — you're ingesting dozens of data types from hundreds of suppliers and normalizing them into one source of truth. That's the problem I'm best at, applied at a scale I haven't worked at yet."

Q33. "Tell me about a time you found a significant data quality issue."

Model answer (adapt with your own specifics): - **S:** During a quarterly refresh for a self-funded employer client, PMPM came in about 12% below the prior quarter with no obvious clinical explanation. - **T:** Client Success had a QBR in four days. I needed to determine whether it was a real trend or a data problem before that meeting. - **A:** I decomposed PMPM into numerator and denominator separately. Member months were flat, so it was the numerator. I broke allowed amount down by service category and by claim type and found the drop was concentrated entirely in institutional claims. Comparing incurred-month totals across the two file deliveries, I found the newest month was materially incomplete — but so was the prior month, which meant it wasn't just normal lag. I traced it back and found the client's clearinghouse had changed their file cut, so a batch of institutional claims fell outside the extract window. I documented the gap by incurred month, quantified the missing dollars, and worked with the client's data team to get a corrected extract. - **R:** We caught it before the QBR instead of after. I then added two permanent monitors — dollar variance by incurred month vs the prior delivery, and a claim-type mix check — which have since caught similar extract issues on two other clients. The broader lesson I took is that a suspicious result should always be decomposed into its parts before you accept it as a trend.

Why this works: it demonstrates domain knowledge (PMPM decomposition, lag, claim types), analytical method, client awareness, and a systemic fix rather than a one-off.

Q34. "Tell me about a time you made a mistake."

Do: pick a real, moderate-stakes mistake. Own it without hedging. Spend most of the answer on detection, correction, communication, and the control you added. **Don't:** pick a fake weakness, blame anyone, or pick something catastrophic and unresolved.

Frame: "*I built a report where a left join to a provider table fanned out because the provider dimension had duplicate NPIs with different taxonomy codes — it inflated utilization counts by about 8%. I caught it during a self-review against a control total before it went to the client, but not before I'd already shared a draft internally. I flagged it immediately, corrected it, and added a grain assertion — a check that row count after a join equals row count before — as a standard step in my review checklist. I'd rather be the person who raises their own error early than the person who hopes nobody notices.*"

Q35. "Tell me about working with a difficult stakeholder."

Avoid "difficult person." Reframe as competing objectives.

> "It wasn't personality — Client Delivery had committed to a go-live date, and the data wasn't clean enough to support it. Instead of just saying no, I quantified exactly what was at risk: 6% of claims were orphaned to members with no eligibility record, representing about 11% of total allowed dollars. I laid out three options — delay two weeks for the corrected eligibility file, go live with a documented caveat and suppress the affected reports, or go live fully and accept that PMPM would be understated. Putting the tradeoff in the client's language rather than mine turned it from a conflict into a decision. We went with option two and closed the gap the following cycle."

Q36. "Tell me about explaining something technical to a non-technical audience."

Key move: lead with the **so-what**, use their vocabulary, use one analogy, offer detail rather than forcing it.

"I told the client 'your reported cost is going to look lower than reality for the last three months, and it will drift up as more claims come in' — rather than 'we have incomplete IBNR runout.' I used the analogy of counting credit card spending before the statement closes. Then I showed a simple chart of the same incurred month restated across three deliveries so they could see the pattern. They stopped asking about it because they understood it, instead of just accepting it."

Q37. "Why do you want this role?"

Hit three notes: **the problem, the company, the trajectory.**

"Three reasons. First, the problem: I'm most useful in the messy layer between raw client data and a trustworthy analytic dataset, and that's what this role is. Second, the scale: 35+ data types, 1,500+ formats, 350+ suppliers — I've solved that problem at single-client scale and I want to solve it as a repeatable system. Third, the stakes: in value-based care, data errors move real money in provider contracts. I'd rather work where accuracy actually matters than where it's a nice-to-have."

Q38. "Where do you see yourself in 3–5 years?"

"Deeper, not just higher. I'd want to become the person who owns a domain end-to-end — the go-to for a data source or a client segment — and to be mentoring newer analysts on healthcare data specifically, because it's a domain where the learning curve is real and hard to shortcut. If a lead or manager path opened up I'd be interested, but I'd want to earn it by being technically indispensable first."

Q39. "How do you handle competing priorities?"

Give a **framework**, then an example.

"I triage on two axes: is this blocking someone else, and what's the cost of it being late or wrong. Client-facing production issues beat internal deadlines. A blocking issue for a teammate beats my own non-blocking work. Then I make the tradeoff visible rather than silently deprioritizing — I'll tell my manager 'if I take the client escalation, the mapping doc slips to Thursday, is that the right call?' Most of the friction I've seen on data teams comes from people absorbing conflicts silently instead of surfacing them."

Q40. "What's your biggest weakness?"

Real, bounded, actively managed.

"I over-invest in validating before I share. It comes from claims work where a wrong number can move contract dollars, but it means I've sometimes sat on a directionally-correct finding for a day longer than necessary when the stakeholder would have been fine with a caveated early read. I've been deliberately working on shipping a 'here's what I'm seeing, here's my confidence level, here's what I still need to check' update earlier in the process."

Q41. "Why are you leaving your current role?" / "Explain this gap."

Forward-looking, never negative.

"I've gotten a lot out of [current role], particularly [specific skill]. What I'm looking for now is [scale / healthcare depth / VBC exposure], and that isn't on the roadmap where I am. This role is a direct match for where I want to go deeper."

PART 6 — SITUATIONAL / SCENARIO QUESTIONS

These are the highest-signal questions for this role. Think out loud; state assumptions; ask clarifying questions.

Q-S1. "A client says their PMPM report is wrong. It's 3pm, they want an answer today. What do you do?"

Answer with a structured triage — do NOT jump to a cause:

1. **Clarify the claim.** Wrong how — too high, too low, versus what benchmark? Their own internal number, last quarter's report, or a gut feeling? Which population, which time period, which metric definition? Half of "the data is wrong" turns out to be a definitional mismatch.
2. **Decompose.** PMPM = cost ÷ member months. Test each side independently. Is the numerator off, the denominator off, or both?
3. **Denominator checks:** member month count vs prior period; duplicate/overlapping eligibility spans; did a large group get added or termed; is the coverage-type filter right (medical vs medical+Rx).
4. **Numerator checks:** completeness by incurred month (claims lag is the #1 culprit for "too low"); duplicate or non-deduped claim versions (#1 culprit for "too high"); allowed vs paid vs billed mismatch; a single high-cost claimant; service category mix shift.
5. **Compare to source.** Reconcile our totals to the client's own control totals from their financials.
6. **Communicate on the clock.** Even if unresolved, I'd send an update by end of day: what I've ruled out, what I'm still checking, and when they'll hear next. Silence is what actually damages the relationship.
7. **Fix forward.** Whatever the root cause, add a monitor so it's caught automatically next time.

Closing line: "*And I'd resist the pressure to give a confident answer I can't support. 'I've ruled out the denominator, I'm now testing claims completeness, I'll have an answer by 10am' is a better answer than a guess.*"

Q-S2. "You're onboarding a new client. Their claims file has 5% of records with member IDs that don't exist in the eligibility file. What now?"

1. **Quantify properly** — 5% of *records*, but what % of *dollars*? And is it concentrated (one group, one date range, one claim type) or scattered? Concentration points to a systematic cause; scatter points to a matching problem.
2. **Hypothesize causes in order of likelihood:** - Eligibility file has shorter history than claims (very common — they sent 24 months of claims and 12 of eligibility) - ID format mismatch: leading zeros stripped, suffixes appended (`123456-01`), or claims carry subscriber ID while eligibility carries member ID - Terminated members with runout claims after coverage ended (legitimate!) - Members from an acquired group or a different plan not included in the extract - COB / other-payer claims
3. **Test cheaply:** try a fuzzy match on ID after normalization; try matching on DOB + name; check whether orphans cluster before the eligibility start date.
4. **Escalate with specifics, not vagueness.** Go to the client with: "5% of claims, 11% of dollars, 80% of them concentrated in Jan–Jun 2024 which is before your eligibility file starts — can you extend the eligibility history?"
5. **Decide the interim handling** and *document it*: exclude orphans from PMPM (protects the denominator integrity) but retain and flag them so they're not lost, and disclose the exclusion in the report footnote.

Senior signal: "*What I won't do is quietly create synthetic eligibility records to make the join work. That makes the report look clean and the number wrong.*"

Q-S3. "Two clients define 'Emergency Room visit' differently. How do you handle it?"

"This is a canonical vs client-specific tension. I'd handle it in layers.

The **canonical layer** has one standard, documented definition — say, revenue codes 0450–0459 and 0981, or POS 23, deduped to one visit per member per facility per date, excluding visits that result in an inpatient admission. Everything in the platform computes off that.

Then a **client-specific reporting layer** can apply their variant on top, clearly labeled. The critical rule is that the client's definition never overwrites the canonical one, because the moment it does, cross-client benchmarking is destroyed — and benchmarking is a big part of the product's value.

Practically I'd bring both to the Data Manager, document each definition and the delta between them, and quantify the impact so the client can see what the difference actually is. Often when a client sees 'your definition produces 8% more visits and here's why,' they adopt the standard. And I'd never let a definitional difference live only in someone's head or in a query — it goes in the metric dictionary."

Q-S4. "You're 2 days from a client go-live and you find a mapping error affecting 15% of claims. What do you do?"

1. **Assess immediately:** which claims, what dollar impact, which downstream reports, does it over- or under-state
2. **Escalate immediately** — same hour, to the reporting manager and Client Delivery. Do not sit on it hoping to fix it quietly.
3. **Bring options, not just a problem:** - Fix and retest (how long, realistically, with buffer) - Go live with the affected reports suppressed and a documented caveat - Delay go-live
4. **Recommend one** with reasoning. If it materially misstates cost, my recommendation is delay or suppress — a client discovering it later costs far more trust than a delay.
5. **Post-mortem:** why didn't validation catch it? Add the check.

The line to say: "*The instinct to hide a problem two days before go-live is the single most expensive instinct in this line of work. A missed deadline is recoverable; a client finding an error we knew about is not.*"

Q-S5. "Your manager asks for an analysis you think is methodologically flawed. What do you do?"

"I'd ask questions first, because they may know something I don't — a client commitment, a contractual definition, a prior precedent. If after that I still think it's flawed, I'd raise it concretely and privately: 'If we compare these two provider groups without risk adjustment, Group B looks 20% more expensive, but their average risk score is 30% higher — so we'd be reporting a difference in population, not performance.' Concrete beats abstract.

Then I'd offer the alternative and, where feasible, run both so the difference is visible rather than theoretical. And if they still want it their way, I'd do it, document the limitation in the deliverable itself, and move on. Disagree and commit — but the caveat goes in the output, not just in my head."

Q-S6. "During analysis you find something the client won't want to hear — their VBC program isn't saving money. What do you do?"

"Report it — but do the work to make sure it's true first, because a negative finding gets scrutinized ten times harder than a positive one. I'd validate risk adjustment, check for claims lag understating or overstating the comparison, verify the comparison group is appropriate, check for regression to the mean, and confirm attribution is stable across periods.

Then I'd deliver it with context rather than as a verdict: here's what we found, here's the confidence level, here's what's driving it, and here's where the opportunity actually is. Almost always a 'no savings overall' finding contains a 'but these three service categories improved and this one drove the loss' story, which is far more useful to them.

I'd also route it through the reporting manager and Client Success before it lands, so nobody's blindsided. Delivering hard findings well is a relationship skill, not just an analytic one."

Q-S7. "A client emails you directly asking for an ad hoc pull, bypassing the normal process. It's outside scope."

"I'd acknowledge fast and helpfully — responsiveness is most of client perception. Then I'd assess: if it's 20 minutes and low risk, I'd likely just do it and inform my manager and Client Success so the relationship stays coordinated. If it's substantial, out of scope, or touches PHI beyond what's normally shared, I'd loop in the Data Manager and Client Success before committing, and reply to the client with 'I can help with this — let me confirm timing with the team and come back to you today.'

What I'd avoid is either extreme: silently absorbing unlimited scope, or being bureaucratic at a client who just needs a number. And I'd never send data outside the agreed channel or scope without confirming it's within the BAA and the client's data agreement."

Q-S8. "You've inherited an undocumented ETL process from someone who left. It breaks. Go."

1. **Stabilize first** — what broke, what's the client impact, is there a last-known-good output I can reuse to buy time
2. **Read the logs and error output** before reading code
3. **Reverse-engineer the flow** by tracing data lineage backwards from the output table, not forwards from the source
4. **Diff the inputs** — most "the pipeline broke" is actually "the input changed": a new column, a changed delimiter, a code value we've never seen, a file that arrived empty
5. **Fix minimally** to restore service; resist refactoring under pressure

6. **Then document it** — flow diagram, field mapping, known quirks, runbook — and add the monitoring that would have caught this
7. **Communicate throughout** to whoever's waiting

Add: "*And I'd treat the documentation as a deliverable, not an afterthought. The reason this incident hurt is that someone left without it.*"

Q-S9. "How do you prioritize when three clients escalate at once?"

> "I'd sort on impact and reversibility. Anything that's actively producing a *wrong number a client is making decisions on* goes first — that's irreversible harm. Then anything blocking a contractual deliverable or go-live. Then everything else.
>
> Then I'd communicate the sequencing immediately to all three channels rather than going quiet on two of them — an honest 'I'm on the Acme issue first, I'll be with you by 2pm' is far better than silence. And I'd tell my manager the moment I think three genuinely can't be done by one person today, because that's their call to make, not mine to absorb."

Q-S10. "There can be significant overtime beyond 9–6, and it's not separately compensated. Are you comfortable with that?"

This has actually been asked here. Be honest and composed — don't over-promise, don't get defensive.

> "I'm used to the rhythm of client-driven work — go-lives, month-end, and escalations don't respect a schedule, and I'll put in what a launch or a client issue needs. What I'd want to understand is whether that's cyclical around implementations and closes, or a steady baseline, because those are different things to plan around. Can you tell me what a typical week and a peak week look like for this team?"

Turning it into a question is the move. It's honest, non-committal, and gets you real information about the role.

Q-S11. "Walk me through how you'd validate that our ETL correctly loaded a client's claims file." *(Likely case-study question)*

Answer as a checklist with reasoning: - **Row-level:** source record count = target record count + documented rejects. Every reject categorized, none silently dropped. - **Financial:** total billed / allowed / paid by incurred month reconciles to the client's control totals within tolerance - **Key integrity:** distinct claim count matches; no unintended duplicates on (claim_id, line_num); no orphan claims post-load - **Field fidelity:** spot-check a stratified sample end-to-end — a high-dollar claim, a $0 claim, a reversal, a claim with unusual characters in the provider name, one from each claim type - **Code sets:** % of ICD/CPT/revenue codes not found in reference tables (should be near zero; anything else means a code set version or formatting issue) - **Distributions:** compare value distributions source vs target — a shifted distribution means a transformation bug even when counts match - **Nulls:** null rate per column vs prior load - **Boundary dates:** min and max service dates match expected extract window - **Reproducibility:** re-run the load and confirm identical output (idempotency)

Close: "*And all of this should be automated and produce a signed-off reconciliation report per load, not be a manual eyeball. Manual validation doesn't survive the third client.*"

PART 7 — QUESTIONS YOU ASK THEM

Asking sharp questions is the fastest way to look senior. Pick 4–6.

About the work 1. When a new client's data comes in, what's the typical onboarding timeline, and where does it most often go wrong? 2. How much of the standardization is codified in configuration versus done analyst-by-analyst? Where are you on that journey? 3. What does the canonical schema look like at a high level — how many core entities, and how stable is it? 4. What's the split between new client implementation work and ongoing production support and escalations? 5. How do you handle client-specific metric definitions without fragmenting the benchmark?

About the team 6. How is the data team structured — how do Data Analysts, Data Managers, and Client Delivery interact day to day? 7. What does the first 90 days look like? What would a successful first six months mean to you? 8. What distinguishes someone doing this job well from someone doing it acceptably?

About the tech 9. What does the pipeline stack look like end to end — where does the proprietary tooling sit relative to AWS-native services? 10. How mature is the automated data quality and monitoring layer? Is that something this role would help build?

About context 11. How has the IQVIA acquisition changed the roadmap or the data assets available to the team? 12. What's the biggest data challenge the team is facing right now?

Never ask in round one: salary, PTO, remote flexibility (the JD is explicit about 3–4 days onsite — don't negotiate it in the interview).

PART 8 — PREP PLAN & FINAL CHECKS

7-day plan

Day	Focus
1	Part 2 domain primer. Learn relationship codes, X12 transactions, code sets, and the billed/allowed/paid chain cold.
2	SQL Q1–Q9. **Write them by hand**, don't just read. Member months and claim dedupe must be automatic.
3	Redshift + AWS (Q10–Q15). Say the distkey/sortkey answer out loud until it's fluent.
4	ETL, data quality, schema mapping (Q16–Q20). This is the heart of the JD.
5	Write your 6 STAR stories out in full. Then compress each to 2 minutes and rehearse aloud.
6	Situational questions. Practice thinking out loud — the process matters more than the answer.
7	Company research (cedargate.com/platform, recent news, IQVIA context). Finalize your questions. Light review only.

Non-negotiables before you walk in

- ▢ Can explain member months and PMPM without notes, including edge cases
- ▢ Can write the member-months SQL from memory
- ▢ Can name the claim-dedupe problem and three ways to solve it
- ▢ Know relationship codes 18 / 01 / 19 / G8
- ▢ Can explain 837P vs 837I and why it matters for counting admissions
- ▢ Can explain claims lag/IBNR and why recent months look artificially cheap
- ▢ Have a Redshift distkey/sortkey answer ready
- ▢ Know that Redshift doesn't enforce PK/unique constraints
- ▢ 6 STAR stories, rehearsed, each with a quantified result
- ▢ 5 questions to ask them, written down
- ▢ Can say what Cedar Gate does in two sentences

Interview-day mechanics

- **Think out loud** on technical problems. They're evaluating reasoning, not recall.
- **Ask clarifying questions** before solving. Jumping straight to a query is a junior tell.
- **State your assumptions** explicitly: "I'm assuming one row per claim line and that allowed is populated on denied claims as zero."
- **Quantify everything** in behavioral answers. "Reduced errors" is noise; "cut orphan claims from 5% to 0.3%" is signal.
- **Volunteer edge cases.** In healthcare data, the person who mentions leading zeros, retro-terms, and claim reversals unprompted is the person who has actually done the job.
- **If you don't know something:** "I haven't worked with that specific one. Here's the closest thing I have done, and here's how I'd approach learning it." Never bluff a healthcare definition — they will know.
- **Logistics:** Burlington is 128/Route 3 — confirm whether it's onsite, and if so, plan for traffic. Have the residency/commutability question answered cleanly.

Red flags to defuse if they apply to you

If you have	Say
< 2 yrs healthcare claims	Lead with the specific claims work you *have* done, name the data types, and be concrete about volume and complexity. Don't apologize.
No Redshift, but other MPP/SQL	"I've worked in [Snowflake/Postgres/SQL Server]. The concepts transfer — columnar storage, distribution and sort strategy, MPP join mechanics. I've read up on Redshift specifics like distkey selection and the unenforced-constraint behavior."
Commute concerns	Have a clean, confident answer. Don't try to renegotiate onsite days in the interview.

Prepared for interview readiness. Adapt all STAR stories with your own specifics — the structures are templates, the content must be yours.